

Neural ODEs

Exercise 1: Implementing ODE solvers

In the first part of this exercise, we will familiarize ourselves with ODEs by implementing our own solvers in NumPy.

Toy-problem: Lotka-Volterra

For the purpose of this exercise, we are simulate the non-linear, time invariant "*predator-prey equations*".

The Lotka-Volterra equations, also known as the predator-prey equations, are a pair of first-order nonlinear differential equations, frequently used to describe the dynamics of biological systems in which two species interact, one as a predator and the other as prey.

They are defined as

$$\begin{aligned}\frac{\partial u_1}{\partial t} &= \alpha u_1 - \beta u_1 u_2 \\ \frac{\partial u_2}{\partial t} &= \gamma u_1 u_2 - \delta u_2\end{aligned}$$

where u_1 is the population of prey (e.g. rabbits) and u_2 the population of predators (e.g. foxes) in thousands.

For this first exercise, we are going to assume that the dynamics are fully known with parameters

$$\alpha=1.5, \beta=1, \gamma=3, \delta=1$$

and the initial condition

$$u_0 = u(t_0) = \begin{pmatrix} 1 \\ 1 \end{pmatrix}$$

By numerically integrating this initial value problem (IVP), we obtain a trajectory $u(t)$.

```
In [1]: import matplotlib.pyplot as plt
import numpy as np
from numpy.typing import ArrayLike
from scipy.integrate import solve_ivp # import ODE solver from SciPy

# %matplotlib widget # uncomment to turn on interactive plots
```

```
In [2]: n_states = 2
```

```

n_steps = 501 # for plotting

# Define time interval of integration
t0 = 0.0
te = 15.0
tspan = (t0, te)
t_traj = np.linspace(t0, te, n_steps)

# Initial condition
u0 = np.array([1.0, 1.0])

# Define dynamics
def f_true(
    _t: float,
    u: ArrayLike,
) -> ArrayLike:
    return np.array([1.5 * u[0] - 1.0 * u[0] * u[1], 3.0 * u[0] * u[1] -

```

We obtain a solution using scipy's `odeint` function:

```
In [3]: sol = solve_ivp(f_true, (t0, te), u0, t_eval=t_traj)
```

The matrix `sol.y` contains the state $u(t)$ and `sol.t` the evaluated time steps, matching `t_traj`:

```
In [4]: sol
```

```

Out[4]: message: The solver successfully reached the end of the integration in
          terval.
          success: True
          status: 0
           t: [ 0.000e+00  3.000e-02 ...  1.497e+01  1.500e+01]
           y: [[ 1.000e+00  1.014e+00 ...  6.362e-02  6.496e-02]
                [ 1.000e+00  1.063e+00 ...  8.118e-01  7.924e-01]]
          sol: None
          t_events: None
          y_events: None
          nfev: 182
          njev: 0
          nlu: 0

```

Let's visualize this trajectory:

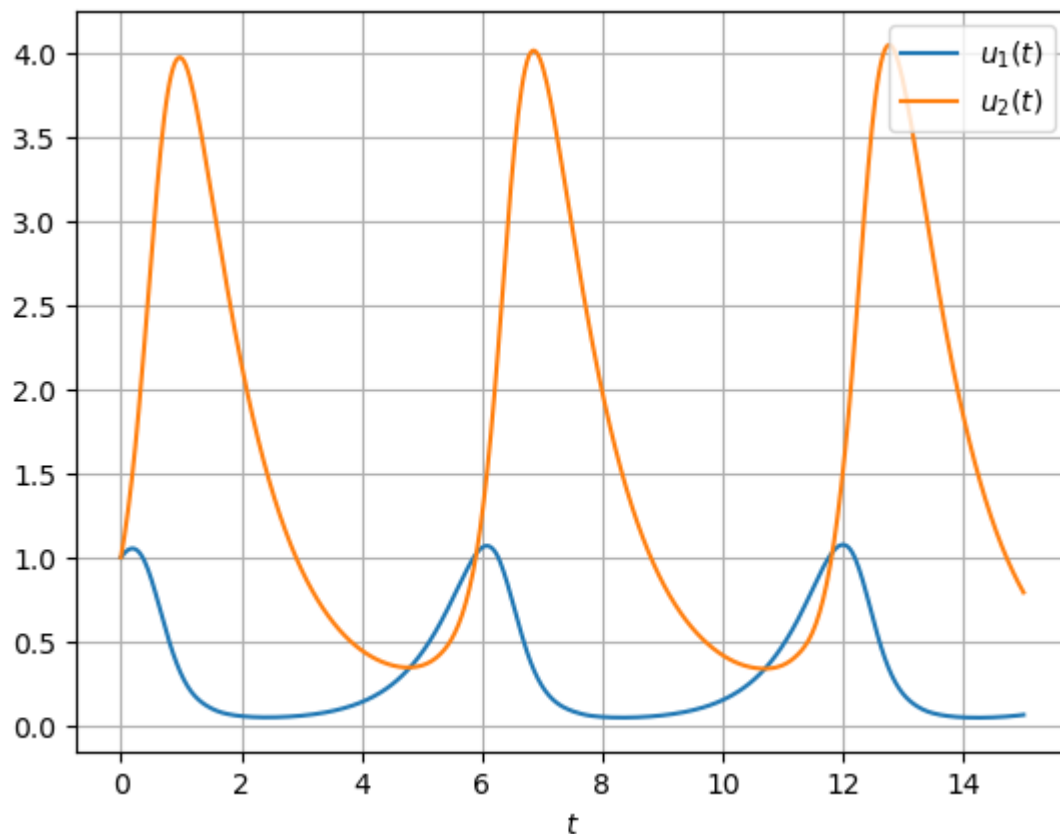
```

In [5]: fig, ax = plt.subplots()
          u_traj = sol.y

          # Plot trajectory that generated data
          ax.plot(t_traj, u_traj[0, :], label="$u_1(t)$")
          ax.plot(t_traj, u_traj[1, :], label="$u_2(t)$")

          ax.legend(loc="best")
          ax.set_xlabel("$t$")
          ax.grid()

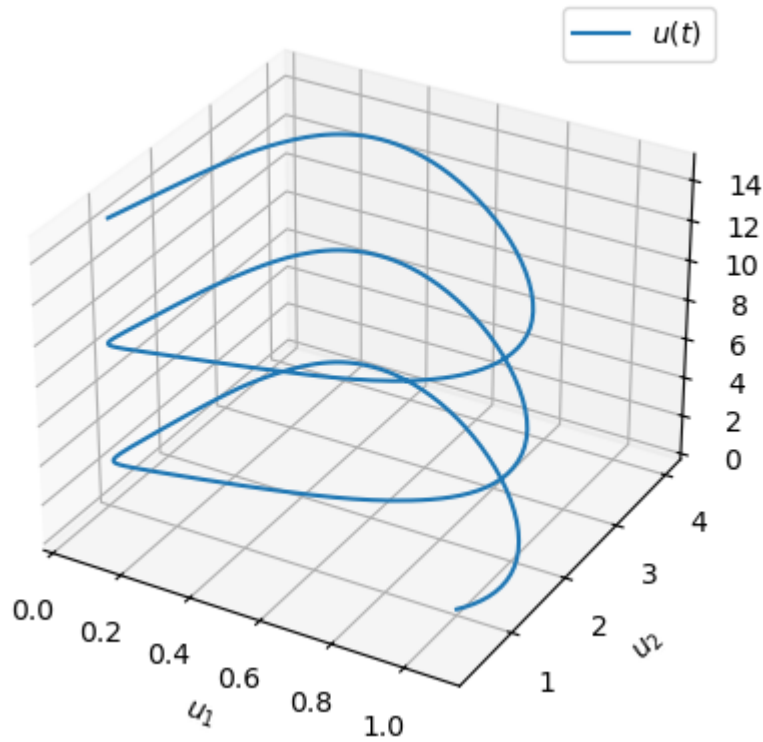
```



We can see that the populations of the predators and prey oscillate over time and keep each other in balance.

Since we are dealing with a 2-dimensional state evolving over time, we can also visualize the trajectory in 3D:

```
In [6]: ax = plt.figure().add_subplot(projection="3d")  
  
ax.plot(u_traj[0, :], u_traj[1, :], t_traj, label="$u(t)$")  
  
ax.legend(loc="best")  
ax.set_xlabel("$u_1$")  
ax.set_ylabel("$u_2$")  
ax.set_zlabel("$t$")  
ax.grid()
```



Implementing ODE solvers

Let's naively implement our own solvers in NumPy, replicating the `solve_ivp` interface.

For this purpose we first introduce an abstract parent class for ODE solvers. Child classes now only have to implement the `step` function.

```
In [7]: from abc import ABC, abstractmethod # abstract base classes

class ODESolver(ABC):
    @abstractmethod
    def step(self, f, t, u):
        """Apply a single integration step.
        Takes in f, tk and uk and returns tk+1, uk+1.
        """
        pass

    def __call__(self, f, tspan, u0):
        """Solve initial value problem, similar to SciPy's solve_ivp."""
        t0, te = tspan
        t = t0
        u = u0

        ts = [t]
        us = [u]
        while t < te:
            t, u = self.step(f, t, u)
            us.append(u)
            ts.append(t)

        us = np.stack(us, axis=-1)
```

```
return us, ts
```

Task 1: Forward Euler method (10 P)

Using a constant step size Δt , a single step of the forward Euler method can be implemented as:
$$\begin{aligned} t_{k+1} &= t_k + \Delta t \\ u_{k+1} &= u_k + \Delta t \cdot f(t_k, u_k) \end{aligned}$$

```
In [8]: class EulerSolver(ODESolver):

    def __init__(self, dt):
        super().__init__()
        self.dt = dt

    def step(self, f, t, u):
        ## INSERT YOUR SOLUTION HERE
        u_next = u + self.dt * f(t, u)
        t_next = t + self.dt
        ## END YOUR SOLUTION
        return t_next, u_next
```

Using a step size of $dt=0.01$, we obtain a numerical approximation to the solution of the IVP:

```
In [44]: euler = EulerSolver(0.1)
u_euler, t_euler = euler(f_true, tspan, u0)
u_euler
```

```
Out[44]: array([[1.00000000e+00, 1.05000000e+00, 1.08150000e+00, 1.08604230e+00,
1.05506315e+00, 9.82434736e-01, 8.68255635e-01, 7.22335044e-01,
5.64069061e-01, 4.16182659e-01, 2.95194583e-01, 2.06145355e-01,
1.45024277e-01, 1.04496023e-01, 7.78289366e-02, 6.01322028e-02,
4.81987743e-02, 4.00100982e-02, 3.43074579e-02, 3.03007492e-02,
2.74871643e-02, 2.55416951e-02, 2.42513236e-02, 2.34751681e-02,
2.31200274e-02, 2.31251498e-02, 2.34526143e-02, 2.40811934e-02,
2.50024215e-02, 2.62180966e-02, 2.77387372e-02, 2.95826994e-02,
3.17757657e-02, 3.43510888e-02, 3.73494159e-02, 4.08195500e-02,
4.48190215e-02, 4.94149587e-02, 5.46851548e-02, 6.07193356e-02,
6.76206379e-02, 7.55073121e-02, 8.45146645e-02, 9.47972588e-02,
1.06531393e-01, 1.19917872e-01, 1.35185081e-01, 1.52592369e-01,
1.72433735e-01, 1.95041774e-01, 2.20791815e-01, 2.50106128e-01,
2.83457975e-01, 3.21375151e-01, 3.64442434e-01, 4.13302044e-01,
4.68650644e-01, 5.31230607e-01, 6.01811878e-01, 6.81158575e-01,
7.69970880e-01, 8.68786881e-01, 9.77819375e-01, 1.09668708e+00,
1.22397527e+00, 1.35652571e+00, 1.48831459e+00, 1.60876329e+00,
1.70045813e+00, 1.73686747e+00, 1.68245580e+00, 1.50129356e+00,
1.18306629e+00, 7.82245516e-01, 4.19750886e-01, 1.90561947e-01,
8.30743193e-02, 3.87564908e-02, 2.00697175e-02, 1.15141420e-02,
7.22934635e-03, 4.90349541e-03, 3.55221825e-03, 2.72226774e-03,
2.18955417e-03, 1.83615794e-03, 1.59660276e-03, 1.43283379e-03,
1.32187939e-03, 1.24945217e-03, 1.20647951e-03, 1.18714809e-03,
1.18776447e-03, 1.20607274e-03, 1.24083741e-03, 1.29158565e-03,
1.35844836e-03, 1.44206503e-03, 1.54353121e-03, 1.66437596e-03,
1.80656156e-03, 1.97250086e-03, 2.16508949e-03, 2.38775188e-03,
2.64450033e-03, 2.94000770e-03, 3.27969431e-03, 3.66983044e-03,
4.11765612e-03, 4.63152020e-03, 5.22104151e-03, 5.89729483e-03,
6.67302540e-03, 7.56289604e-03, 8.58377158e-03, 9.75504607e-03,
1.10990192e-02, 1.26413288e-02, 1.44114489e-02, 1.64432608e-02,
1.87757108e-02, 2.14535654e-02, 2.45282787e-02, 2.80589898e-02,
3.21136682e-02, 3.67704295e-02, 4.21190468e-02, 4.82626862e-02,
5.53199000e-02, 6.34269146e-02, 7.27402569e-02, 8.34397682e-02,
9.57320615e-02, 1.09854486e-01, 1.26079672e-01, 1.44720737e-01,
1.66137244e-01, 1.90742016e-01, 2.19008918e-01, 2.51481734e-01,
2.88784254e-01, 3.31631729e-01, 3.80843781e-01, 4.37358903e-01,
5.02250550e-01, 5.76744752e-01, 6.62238916e-01, 7.60321048e-01,
8.72787790e-01, 1.00165808e+00, 1.14917619e+00, 1.31779175e+00],
[1.00000000e+00, 1.20000000e+00, 1.45800000e+00, 1.78524810e+00,
2.18837978e+00, 2.66220546e+00, 3.18061784e+00, 3.69103287e+00,
4.12177830e+00, 4.40709076e+00, 4.51662810e+00, 4.46495054e+00,
4.29458413e+00, 4.05197140e+00, 3.77379873e+00, 3.48453208e+00,
3.19893865e+00, 2.92530026e+00, 2.66788270e+00, 2.42855291e+00,
2.20777371e+00, 2.00520197e+00, 1.82004665e+00, 1.65128355e+00,
1.49778444e+00, 1.35839464e+00, 1.23197910e+00, 1.11744913e+00,
1.01377707e+00, 9.20003429e-01, 8.35239308e-01, 7.58665922e-01,
6.89532346e-01, 6.27152237e-01, 5.70900022e-01, 5.20206854e-01,
4.74556552e-01, 4.33481645e-01, 3.96559623e-01, 3.63409438e-01,
3.33688288e-01, 3.07088724e-01, 2.83336085e-01, 2.62186293e-01,
2.43424026e-01, 2.26861314e-01, 2.12336600e-01, 1.99714362e-01,
1.88885392e-01, 1.79767917e-01, 1.72309801e-01, 1.66492199e-01,
1.62335195e-01, 1.59906237e-01, 1.59332581e-01, 1.60819589e-01,
1.64677750e-01, 1.71362875e-01, 1.81536548e-01, 1.96158149e-01,
2.16626776e-01, 2.45002991e-01, 2.84359307e-01, 3.39338988e-01,
4.17049694e-01, 5.28482279e-01, 6.90703990e-01, 9.30029038e-01,
1.28588511e+00, 1.81327473e+00, 2.57677263e+00, 3.61968718e+00,
4.88798239e+00, 6.13402631e+00, 6.96011805e+00, 7.14056096e+00,
6.83472062e+00, 6.32158549e+00, 5.76292768e+00, 5.22133301e+00,
4.71723546e+00, 4.25574268e+00, 3.83642881e+00, 3.45687428e+00,
3.11401001e+00, 2.80465450e+00, 2.52573399e+00, 2.27437037e+00,
```

```

2.04791097e+00, 1.84393200e+00, 1.66022997e+00, 1.49480788e+00,
1.34585946e+00, 1.21175309e+00, 1.09101622e+00, 9.82320726e-01,
8.84469279e-01, 7.96382803e-01, 7.17089053e-01, 6.45712203e-01,
5.81463395e-01, 5.23632190e-01, 4.71578831e-01, 4.24727251e-01,
3.82558769e-01, 3.44606395e-01, 3.10449699e-01, 2.79710183e-01,
2.52047111e-01, 2.27153753e-01, 2.04753998e-01, 1.84599307e-01,
1.66465967e-01, 1.50152620e-01, 1.35478035e-01, 1.22279105e-01,
1.10409046e-01, 9.97357710e-02, 9.01404317e-02, 8.15161048e-02,
7.37666115e-02, 6.68054565e-02, 6.05548754e-02, 5.49449799e-02,
4.99129921e-02, 4.54025597e-02, 4.13631452e-02, 3.77494836e-02,
3.45211027e-02, 3.16419036e-02, 2.90797977e-02, 2.68063995e-02,
2.47967755e-02, 2.30292519e-02, 2.14852867e-02, 2.01494154e-02,
1.90092853e-02, 1.80558019e-02, 1.72834217e-02, 1.66906466e-02,
1.62807997e-02, 1.60632114e-02, 1.60550114e-02, 1.62838456e-02,
1.67920265e-02, 1.76429652e-02, 1.89313150e-02, 2.07992995e-02,
2.34636131e-02, 2.72608783e-02, 3.27266143e-02, 4.07365466e-0
2]])

```

Task 2: Runge-Kutta 4 method (20 P)

Let's use a more sophisticated method of integration. The Runge-Kutta methods are a group of IVP solvers, of which the fourth order method is the most widely known.

Using a constant step size Δt , a single step of the Runge-Kutta 4 method can be implemented as:
$$t_{k+1} = t_k + \Delta t$$

$$u_{k+1} = u_k + \frac{\Delta t}{6}(k_1 + 2k_2 + 2k_3 + k_4)$$
 with
$$k_1 = f(u_k, t_k)$$

$$k_2 = f(u_k + \frac{\Delta t}{2}k_1, t_k + \frac{\Delta t}{2})$$

$$k_3 = f(u_k + \frac{\Delta t}{2}k_2, t_k + \frac{\Delta t}{2})$$

$$k_4 = f(u_k + \Delta t k_3, t_k + \Delta t)$$

(Note: By default, SciPy uses the adaptive step size Runge-Kutta 5(4) method RK45 .)

```

In [10]: class RungeKuttaSolver(ODESolver):

    def __init__(self, dt):
        super().__init__()
        self.dt = dt

    def step(self, f, t, u):
        ## INSERT YOUR SOLUTION HERE
        t_next = t + self.dt
        k1 = f(t, u)
        k2 = f(t + self.dt/2, u + self.dt/2 * k1)
        k3 = f(t + self.dt/2, u + self.dt/2 * k2)
        k4 = f(t + self.dt, u + self.dt * k3)
        u_next = u + self.dt/6 * (k1 + 2*k2 + 2*k3 + k4)
        ## END YOUR SOLUTION
        return t_next, u_next

```

```

In [11]: rk = RungeKuttaSolver(0.1)
u_rk, t_rk = rk(f_true, tspan, u0)
u_rk

```

```
Out[11]: array([[1.          , 1.03980898, 1.0533215 , 1.03247374, 0.9729192 ,
 0.87736197, 0.75667975, 0.62692118, 0.50351596, 0.39661594,
 0.31003662, 0.24297911, 0.19240756, 0.15477759, 0.12690464,
 0.10624433, 0.09088345, 0.07942855, 0.07088052, 0.06452773,
 0.05986422, 0.05653036, 0.05427099, 0.05290613, 0.05231084,
 0.05240111, 0.05312429, 0.05445247, 0.05637794, 0.05891025,
 0.06207439, 0.06590968, 0.07046939, 0.0758208 , 0.0820457 ,
 0.08924112, 0.09752047, 0.10701481, 0.11787433, 0.13026999,
 0.14439517, 0.16046729, 0.17872927, 0.1994505 , 0.22292713,
 0.24948111, 0.27945733, 0.31321781, 0.3511315 , 0.3935575 ,
 0.44081854, 0.49316011, 0.55068896, 0.61328172, 0.68045187,
 0.7511611 , 0.8235619 , 0.89467044, 0.96000492, 1.01330873,
 1.0466304 , 1.05121747, 1.0196935 , 0.94939746, 0.84537633,
 0.72044096, 0.59107844, 0.47156791, 0.37023182, 0.28935119,
 0.22727419, 0.18068759, 0.14609213, 0.12047186, 0.10146739,
 0.0873243 , 0.07677199, 0.06890179, 0.06306698, 0.05880782,
 0.05579761, 0.0538048 , 0.05266659, 0.05227071, 0.05254281,
 0.05343772, 0.05493358, 0.05702777, 0.05973434, 0.06308236,
 0.06711508, 0.0718897 , 0.0774775 , 0.08396445, 0.09145208,
 0.10005859, 0.10992027, 0.12119297, 0.13405377, 0.14870265,
 0.16536406, 0.18428827, 0.2057522 , 0.2300594 , 0.25753866,
 0.28854041, 0.32342988, 0.3625753 , 0.40632875, 0.45499612,
 0.50879125, 0.56776702, 0.63171363, 0.70001141, 0.77142394,
 0.84382025, 0.91383195, 0.97649809, 1.02505288, 1.0511814 ,
 1.04623656, 1.00380199, 0.92315664, 0.8117054 , 0.68385455,
 0.55603789, 0.4410785 , 0.34547366, 0.2701492 , 0.21278456,
 0.16990484, 0.1381065 , 0.11455423, 0.09706876, 0.08404455,
 0.07432419, 0.06708156, 0.06172902, 0.05784886, 0.05514434,
 0.05340548, 0.05248524, 0.05228307, 0.05273354, 0.05379855,
 0.05546198, 0.05772609, 0.06060929, 0.06414468, 0.06837944,
 0.07337469, 0.07920575],
 [1.          , 1.229386 , 1.52386092, 1.88711621, 2.30902652,
 2.75981718, 3.19220066, 3.55470273, 3.80959857, 3.9434553 ,
 3.96513809, 3.89633249, 3.76208941, 3.58504076, 3.3831667 ,
 3.16967846, 2.95378066, 2.74161963, 2.53712712, 2.34268044,
 2.15958771, 1.9884329 , 1.82931707, 1.68202547, 1.54614268,
 1.42113194, 1.30638941, 1.20128132, 1.10516911, 1.0174263 ,
 0.93744959, 0.86466595, 0.79853696, 0.73856127, 0.68427582,
 0.63525629, 0.5911171 , 0.55151128, 0.51613044, 0.48470501,
 0.45700505, 0.43284178, 0.41207023, 0.39459335, 0.38036818,
 0.36941471, 0.36182865, 0.35779959, 0.35763655, 0.36180444,
 0.37097558, 0.38610308, 0.40852491, 0.44011087, 0.48346698,
 0.54221141, 0.62132487, 0.72754064, 0.86964473, 1.05835761,
 1.30514593, 1.61900397, 2.00052236, 2.43443121, 2.88533324,
 3.30301153, 3.63815187, 3.85933169, 3.9600531 , 3.95385287,
 3.86418362, 3.71582536, 3.53021004, 3.32393467, 3.10899212,
 2.89363432, 2.68330832, 2.48145095, 2.2900972 , 2.11032221,
 1.94255375, 1.78678997, 1.64274997, 1.50997755, 1.38791249,
 1.27593943, 1.17342114, 1.07972103, 0.99421822, 0.91631734,
 0.84545472, 0.78110219, 0.72276907, 0.67000315, 0.62239099,
 0.57955788, 0.54116767, 0.50692282, 0.4765648 , 0.44987502,
 0.42667668, 0.40683778, 0.39027573, 0.37696421, 0.36694298,
 0.36033203, 0.35735148, 0.35834993, 0.36384451, 0.37457786,
 0.39159907, 0.41637843, 0.45096899, 0.49822981, 0.56212343,
 0.64808358, 0.76339928, 0.9174424 , 1.12133095, 1.38627402,
 1.71961802, 2.11823418, 2.56132426, 3.00827438, 3.40723998,
 3.71248695, 3.89946206, 3.96816986, 3.93597836, 3.82741826,
 3.6663652, 3.47375722, 3.26401852, 3.04826571, 2.83387262,
 2.62564958, 2.42658613, 2.2384094 , 2.06198587, 1.89760406,
 1.74517089, 1.60434741, 1.47464215, 1.35547525, 1.24622241,
```



```
1.14624468, 1.05490864, 0.97159976, 0.89573109, 0.82674864,
0.76413461, 0.70740908]])
```

Visualization

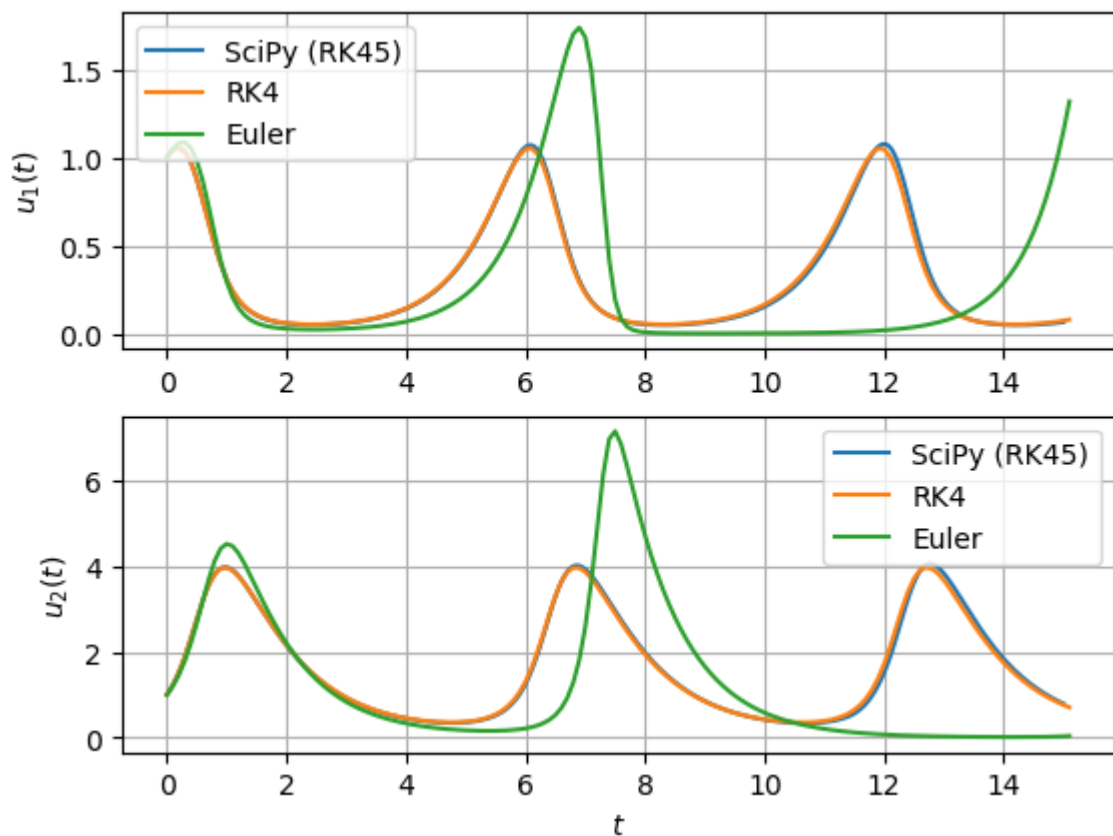
By plotting the results, we observe that the Runge-Kutta 4 method is much more accurate than the simple forward Euler method, which diverges over time as numerical errors accumulate.

```
In [12]: fig, axs = plt.subplots(2, 1)

for i in range(n_states):
    # Plot true trajectory
    axs[i].plot(t_traj, u_traj[i, :], label="SciPy (RK45)")
    # Plot solvers
    axs[i].plot(t_rk, u_rk[i, :], label=r"RK4")
    axs[i].plot(t_euler, u_euler[i, :], label=r"Euler")

    axs[i].set_ylabel(f"$u_{i+1}(t)$")
    axs[i].legend(loc="best")
    axs[i].grid()

axs[n_states - 1].set_xlabel("$t$");
```



Exercise 2: Trajectory Optimization

Dataset generation

Using the dynamics defined in exercise 1, we generate a small dataset of 31 points at equidistant time steps from $t_0=0$ to $t_{\text{end}}=3$:

```
In [13]: te = 3.0
n_data = 31
t_data = np.linspace(t0, te, n_data)

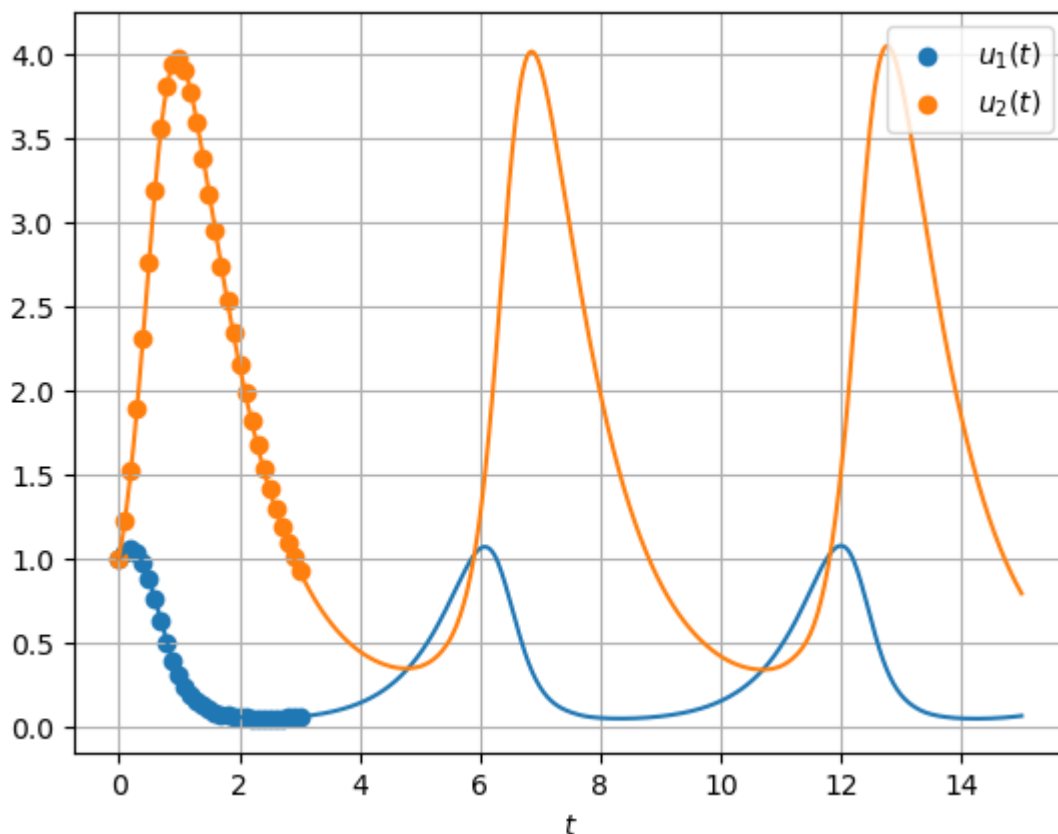
sol_data = solve_ivp(f_true, (t0, te), u0, t_eval=t_data)
u_data = sol_data.y
```

```
In [14]: fig, ax = plt.subplots()

# Plot trajectory that generated data
ax.plot(t_traj, u_traj[0, :], c="C0")
ax.plot(t_traj, u_traj[1, :], c="C1")

# Plot data
ax.scatter(t_data, u_data[0, :], c="C0", label="$u_1(t)$")
ax.scatter(t_data, u_data[1, :], c="C1", label="$u_2(t)$")

ax.legend(loc="best")
ax.set_xlabel("$t$")
ax.grid()
```



Defining a Neural ODE in torchdiffeq

Using PyTorch, we want to learn the ground truth dynamics $f(t, u)$ that generated our dataset (u_data, t_data) by fitting a Neural ODE $f_{\theta}(t, u)$.

To compute gradients of a loss function w.r.t. θ , we import `torchdiffeq`, which implements the *adjoint method*. We can construct an adjoint ODE and integrate it backwards in time using the `odeint_adjoint` function.

```
In [36]: import torch
import torch.nn as nn
import torch.optim as optim
from torchdiffeq import odeint_adjoint as odeint

# Fix random seed
torch.manual_seed(123)

# Automatically use GPU if CUDA is available
device = torch.device("cuda:" + str(0) if torch.cuda.is_available() else
torch.cuda.is_available())
```

Out[36]: False

```
In [34]: # Convert dataset to Torch tensors
u0_torch = torch.tensor([1.0, 1.0]).to(device)
t_torch = torch.tensor(t_data).to(device)
u_target = torch.tensor(u_data.transpose()).to(device)
t_torch.shape
```

Out[34]: torch.Size([31])

We define a small wrapper module that implements a forward function $f_\theta(t, u)$.

```
In [20]: class MyNeuralODE(nn.Module):
def __init__(self, net):
super().__init__()
self.net = net # neural network

def forward(self, _t, u):
return self.net(u)
```

We now need to define a neural network. Usually, these are kept very small, since they will be repeatedly applied during the integration.

High derivatives $\dot{u}=f_\theta(t, u)$ will quickly make the trajectory diverge toward ∞ . To make training more stable, we modify the weights and biases to be very small at initialization:

```
In [21]: net = nn.Sequential(
nn.Linear(2, 16), # input dimensionality of u
nn.Tanh(),
nn.Linear(16, 2), # output dimensionality of u
)

for layer in net.modules():
if isinstance(layer, nn.Linear):
nn.init.normal_(layer.weight, mean=0.0, std=0.05)
nn.init.constant_(layer.bias, val=0.0)

# Define Neural ODE
```

```
neural_ode = MyNeuralODE(net).to(device)
```

Let's integrate the neural ODE and visualize the initialization.

Note that compared to SciPy's `solve_ivp`, the output is transposed
(`(n_steps, n_states)` instead of `(n_states, n_steps)`).

```
In [22]: u_pred = odeint(neural_ode, u0_torch, t_torch).to(device)
         u_pred.size()
```

```
Out[22]: torch.Size([31, 2])
```

Hidden in the following cell are two helper functions

`visualize_prediction_2d` and `3d`, plot the current trajectory of the neural ODE.

```
In [23]: def pred_to_numpy(u):
         return u.detach().cpu().numpy().transpose()

def visualize_prediction_2d(neural_ode):
    u_pred = pred_to_numpy(odeint(neural_ode, u0_torch, t_torch))

    # Compute full neural ODE trajectory
    t_full = torch.linspace(t0, te, 501).to(device)
    u_full = pred_to_numpy(odeint(neural_ode, u0_torch, t_full))

    fig, axs = plt.subplots(2, 1)

    for i in range(n_states):
        # Plot data
        axs[i].scatter(t_data, u_data[i, :], c="C0", label="Data")
        # Plot neural ODE prediction
        axs[i].scatter(t_data, u_pred[i, :], c="C1", label="Prediction")
        # Plot full neural ODE trajectory
        axs[i].plot(t_full, u_full[i, :], c="C1")

        axs[i].set_ylabel(f"$u_{i+1}(t)$")
        axs[i].legend(loc="best")
        axs[i].grid()

    axs[n_states - 1].set_xlabel("$t$")

def visualize_prediction_3d(neural_ode):
    u_pred = pred_to_numpy(odeint(neural_ode, u0_torch, t_torch))

    # Compute full neural ODE trajectory
    t_full = torch.linspace(t0, te, 501).to(device)
    u_full = pred_to_numpy(odeint(neural_ode, u0_torch, t_full))

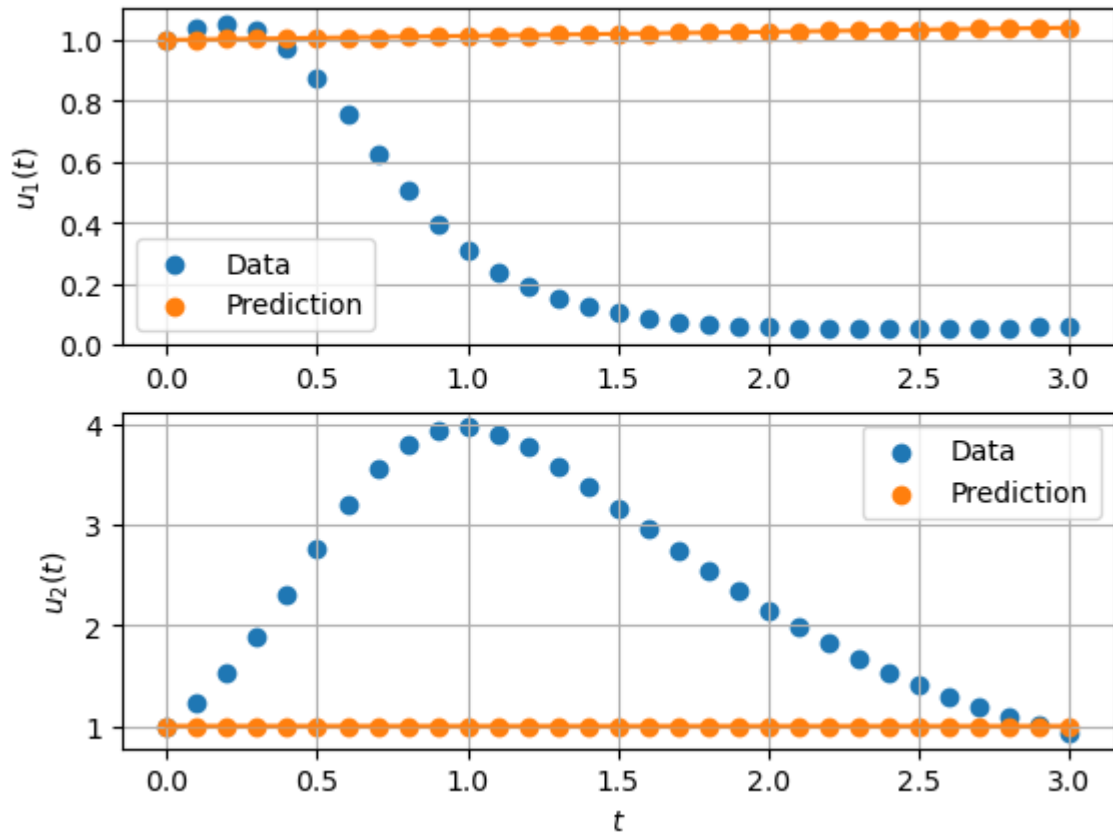
    ax = plt.figure().add_subplot(projection="3d")

    # Plot data
    ax.scatter(u_data[0, :], u_data[1, :], t_data, c="C0", label="Data")
    # Plot neural ODE prediction
    ax.scatter(u_pred[0, :], u_pred[1, :], t_data, c="C1", label="Predict")
    # Plot full neural ODE trajectory
```

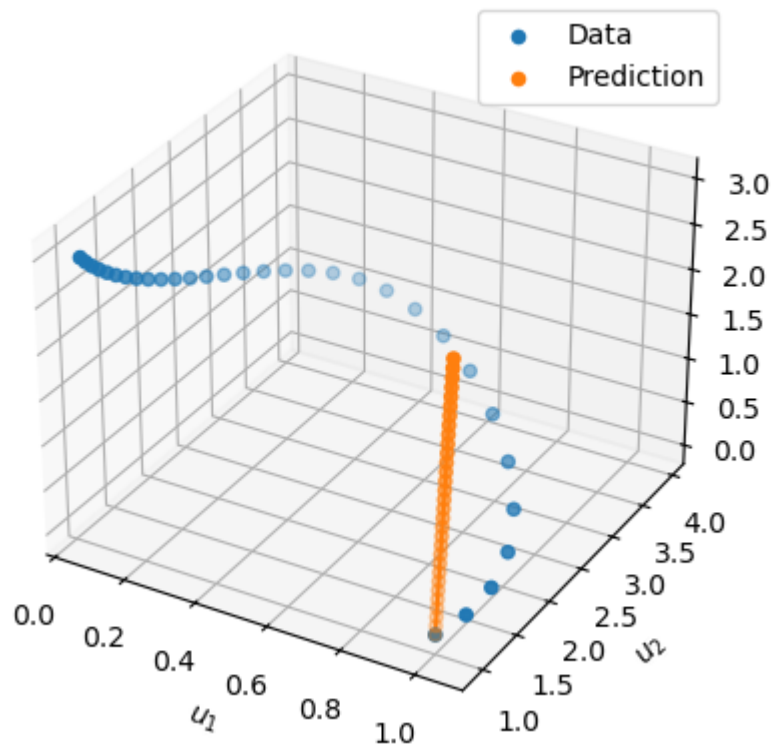
```
ax.plot(u_full[0, :], u_full[1, :], t_full, c="C1")

ax.legend(loc="best")
ax.set_xlabel("$u_1$")
ax.set_ylabel("$u_2$")
ax.set_zlabel("$t$")
ax.grid()
```

In [25]: visualize_prediction_2d(neural_ode)



In [26]: visualize_prediction_3d(neural_ode)



Task 3: Loss function (10 P)

Implement a loss function `loss_fn(u, u_pred)` that implements

$$\mathcal{L}(\theta) = \sum_{k=0}^N \|u_k - \hat{u}_k\|^2$$

where N corresponds to `n_steps`, u_k to the target state vector at time step k and $\hat{u}_k$ to the predicted state vector that results from integrating the neural ODE.

You can assume that both `u` and `u_pred` are of shape `(n_steps, n_states)`. Make sure the output is a scalar value.

```
In [29]: def loss_fn(u, u_pred):
          ## INSERT YOUR SOLUTION HERE
          #print(u.shape, u_pred.shape)
          loss = torch.sum(torch.norm((u - u_pred)**2, dim=1))
          ## END YOUR SOLUTION
          return loss
```

```
In [30]: loss_fn(u_target, u_pred)
```

```
Out[30]: tensor(99.7071, dtype=torch.float64, grad_fn=<SumBackward0>)
```

Task 4: Training (30 P)

Implement the training loop:

- Compute a prediction using `odeint`.

- Compute the loss using the `loss_fn` from task 3 and `u_target`.
- Compute gradients and apply a step of the optimizer.

Optional bonus task: Create an animation of the trajectories during training using `visualize_prediction_2d / 3d` (or your own plotting function) and save it as a GIF.

```
In [39]: import io
import torch
import matplotlib.pyplot as plt
from PIL import Image

class Gif:
    def __init__(self, u_target=None):
        self.frames = []
        self.u_target = u_target

    def frame(self, neural_ode, u0_torch, t_torch):
        with torch.no_grad():
            # Integrate the ODE forward in time to compute the trajectory
            u_pred = odeint(neural_ode, u0_torch, t_torch).cpu().numpy()

            fig, ax = plt.subplots(figsize=(6, 6))

            if self.u_target is not None:
                target = self.u_target.cpu().numpy()
                ax.plot(target[:, 0], target[:, 1], 'g-', label='Target', alp

            ax.plot(u_pred[:, 0], u_pred[:, 1], 'b--', label='Prediction')
            ax.set_title("Phase Portrait Optimization")
            ax.set_xlabel("State 1")
            ax.set_ylabel("State 2")
            ax.legend()

            # Render the canvas to a byte stream to avoid disk I/O bottleneck
            buf = io.BytesIO()
            plt.savefig(buf, format='png', bbox_inches='tight', dpi=100)
            buf.seek(0)

            # Decode the buffer into a PIL Image object and store
            img = Image.open(buf)
            self.frames.append(img.copy())

            plt.close(fig)
            buf.close()

    def save(self, path):
        if not self.frames:
            raise RuntimeError("Empty frames")

        self.frames[0].save(
            path,
            format='GIF',
            save_all=True,
            append_images=self.frames[1:],
            optimize=False,
            duration=100,
            loop=0
```

)

```

In [43]: torch.manual_seed(123) # Fix random seed

optimizer = optim.Adam(neural_ode.parameters(), lr=2e-3)

n_iters = 1000
print_every = 50
gif = Gif(u_target)

for i in range(1, n_iters + 1):
    optimizer.zero_grad()

    ## INSERT YOUR SOLUTION HERE
    u_pred = odeint(neural_ode, u0_torch, t_torch).to(device) # compute p
    loss = loss_fn(u_target, u_pred) # compute loss
    grad = loss.backward() # compute gradients
    optimizer.step() # update optimizer state
    ## END YOUR SOLUTION

    if i == 1 or i % print_every == 0:
        print(f"Step: {i :4d} | Loss: {loss.item() :.6f}")
        gif.frame(neural_ode, u0_torch, t_torch)

gif.save('ode.gif')

```

```

Step:   1 | Loss: 0.865249
Step:  50 | Loss: 0.849435
Step: 100 | Loss: 0.834332
Step: 150 | Loss: 0.822357
Step: 200 | Loss: 0.808889
Step: 250 | Loss: 0.790990
Step: 300 | Loss: 0.765904
Step: 350 | Loss: 0.730398
Step: 400 | Loss: 0.681223
Step: 450 | Loss: 0.616795
Step: 500 | Loss: 0.539158
Step: 550 | Loss: 0.454229
Step: 600 | Loss: 0.369553
Step: 650 | Loss: 0.296775
Step: 700 | Loss: 0.236070
Step: 750 | Loss: 0.186327
Step: 800 | Loss: 0.147790
Step: 850 | Loss: 0.118820
Step: 900 | Loss: 0.093419
Step: 950 | Loss: 0.074991
Step: 1000 | Loss: 0.061196

```

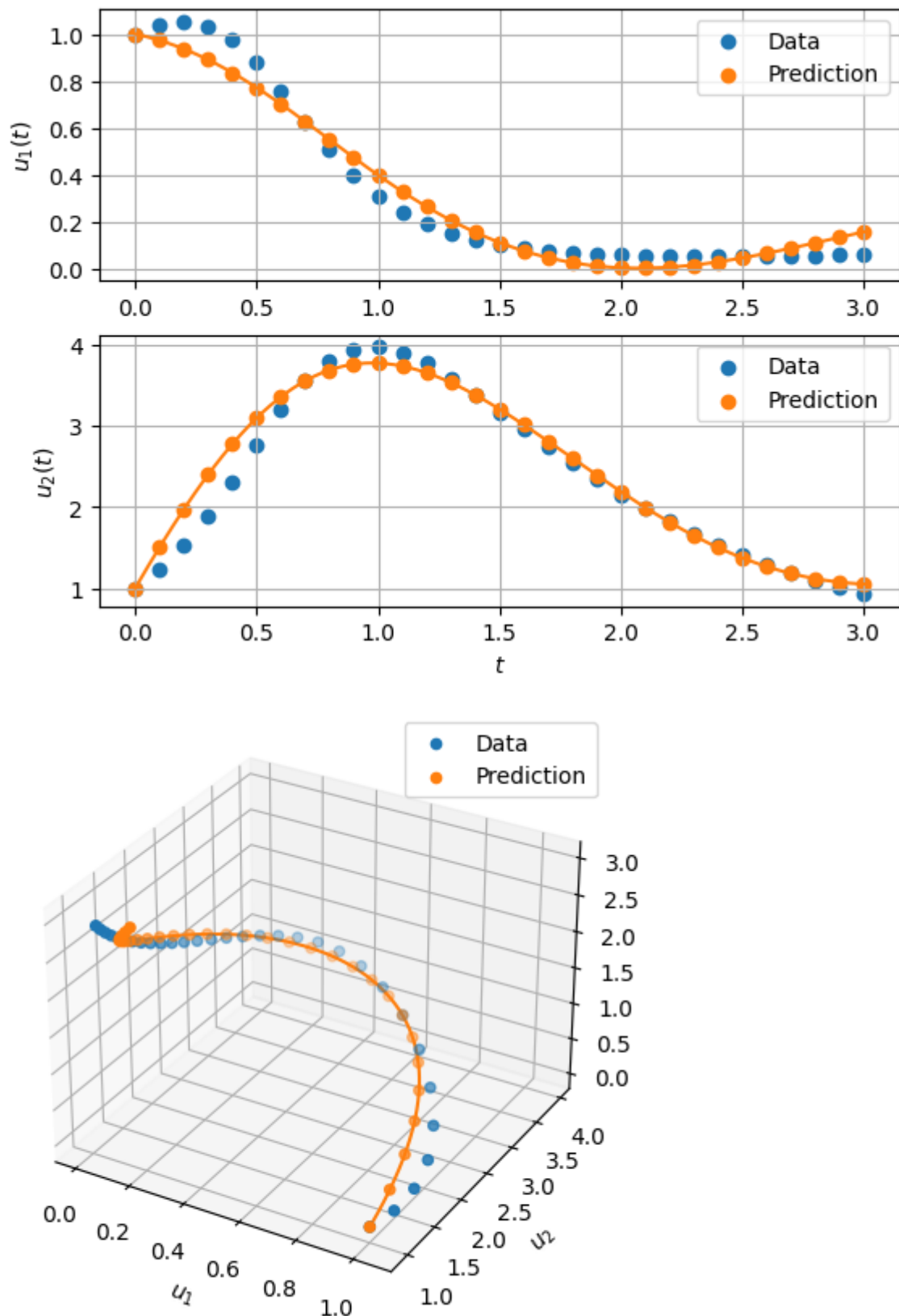
In []:

Let's visualize the resulting fit:

```

In [29]: visualize_prediction_2d(neural_ode)
         visualize_prediction_3d(neural_ode)

```

Since our neural ODE hasn't fully converged yet, you can increase `n_iters`, tweak the learning rate and re-train to obtain a better prediction.

We could now use the trained neural network as a surrogate model. For example, as shown in the lecture, we could apply symbolic regression to recover the Lotka-Volterra equations.